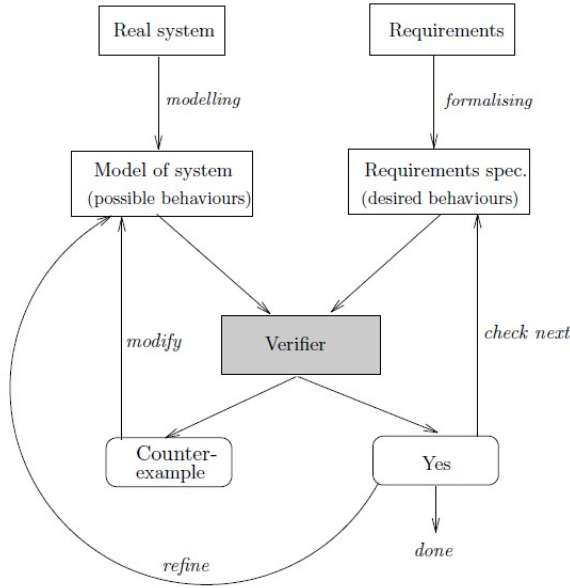




- *Safety* properties state that “something bad never happens”. A system satisfies such a property if it does not engage in the proscribed activity. For instance, a typical safety property for a coffee-machine is that the machine will provide never tea if the user requests coffee.
- *Liveness* properties state that “something good will eventually happen”; to satisfy such a property, the system must engage in some desired activity. An example liveness property for the coffee-machine is that it will provide coffee eventually after a sufficient payment by the user. Stated differently, the user will always be provided coffee at some time after inserting the payment.

Definition 2. (Model for PLTL)

A **PLTL**-model is a triple $\mathcal{M} = (S, R, Label)$ where

- S is a non-empty denumerable set of states,
- $R : S \longrightarrow S$, assigning to $s \in S$ its unique successor state $R(s)$,
- $Label : S \longrightarrow 2^{AP}$, assigning to each state $s \in S$ the atomic propositions $Label(s)$ that are valid in s .

The following definition determines the set of basic formulas that can be stated in propositional linear temporal logic (PLTL, for short).

Definition 1. (Syntax of propositional linear temporal logic)

Let AP be a set of atomic propositions. Then:

1. For all $p \in AP$, p is a formula.
2. If ϕ is a formula, then $\neg\phi$ is a formula.
3. If ϕ and ψ are formulas, then $\phi \vee \psi$ is a formula.
4. If ϕ is a formula, then $X\phi$ is a formula.
5. If ϕ and ψ are formulas, then $\phi U \psi$ is a formula.

The usual boolean operators $\wedge$ (conjunction), $\Rightarrow$ (implication) and $\Leftrightarrow$ (equivalence) are defined as

$$\begin{aligned}\phi \wedge \psi &\equiv \neg(\neg\phi \vee \neg\psi) \\ \phi \Rightarrow \psi &\equiv \neg\phi \vee \psi \\ \phi \Leftrightarrow \psi &\equiv (\phi \Rightarrow \psi) \wedge (\psi \Rightarrow \phi).\end{aligned}$$

The formula true equals $\phi \vee \neg\phi$ and false equals $\neg\text{true}$. The temporal operators G (pronounced “always” or “Globally”) and F (pronounced “eventually” or “Future”) are defined by

$$\begin{aligned}F\phi &\equiv \text{true} U \phi \\ G\phi &\equiv \neg F \neg\phi.\end{aligned}$$

Since true is valid in all states, $F\phi$ indeed denotes that ϕ holds at some point in the future. Suppose there is no point in the future for which $\neg\phi$ holds. Then, the counterpart of ϕ holds at any point. This explains the definition of $G\phi$. In the literature F is sometimes denoted as $\Diamond$ and G as $\Box$. In these lecture notes we will use the traditional notations F and G .

Alternatively, the syntax of PLTL can be given in Backus-Naur Form (BNF, for short) as follows: for $p \in AP$ the set of PLTL-formulas is defined by

$$\phi ::= p \mid \neg\phi \mid \phi \vee \phi \mid X\phi \mid \phi U \phi.$$

The meaning of formulas in logic is defined by means of a satisfaction relation (denoted by $\models$) between a model $\mathcal{M}$, one of its states s , and a formula ϕ . $(\mathcal{M}, s, \phi) \in \models$ is denoted by the following infix notation: $\mathcal{M}, s \models \phi$. The concept is that $\mathcal{M}, s \models \phi$ if and only if ϕ is valid in the state s of the model $\mathcal{M}$. When the model $\mathcal{M}$ is clear from the context we often omit the model and simply write $s \models \phi$ rather than $\mathcal{M}, s \models \phi$.

Definition 3. (Semantics of PLTL)

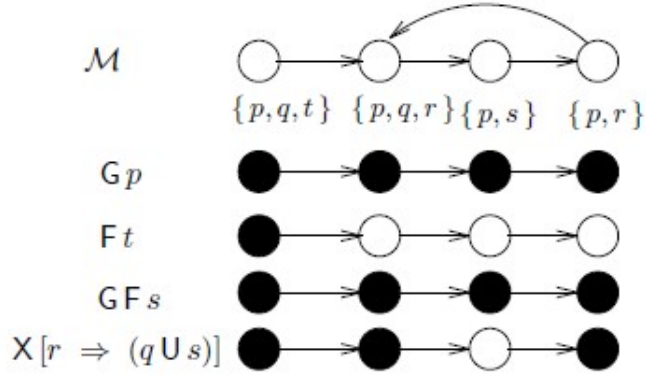
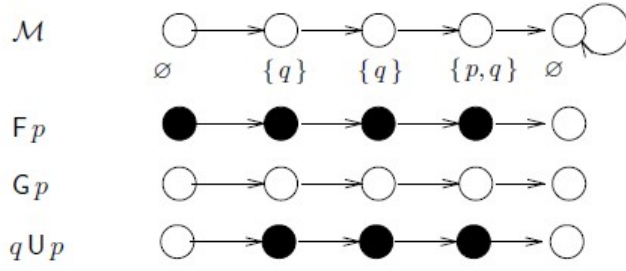
Let $p \in AP$ be an atomic proposition, $\mathcal{M} = (S, R, Label)$ a PLTL-model, $s \in S$, and ϕ, ψ PLTL-formulas. The satisfaction relation $\models$ is defined by:

$$\begin{aligned} s \models p & \quad \text{iff } p \in Label(s) \\ s \models \neg \phi & \quad \text{iff } \neg(s \models \phi) \\ s \models \phi \vee \psi & \quad \text{iff } (s \models \phi) \vee (s \models \psi) \\ s \models X\phi & \quad \text{iff } R(s) \models \phi \\ s \models \phi U \psi & \quad \text{iff } \exists j \geq 0. R^j(s) \models \psi \wedge (\forall 0 \leq k < j. R^k(s) \models \phi). \end{aligned}$$

Here $R^0(s) = s$, and $R^{n+1}(s) = R^n(R(s))$ for any $n \geq 0$.

$$\begin{aligned} s \models F\phi & \\ \Leftrightarrow \{ \text{definition of } F \} & \\ s \models \text{true} U \phi & \\ \Leftrightarrow \{ \text{semantics of } U \} & \\ \exists j \geq 0. R^j(s) \models \phi \wedge (\forall 0 \leq k < j. R^k(s) \models \text{true}) & \\ \Leftrightarrow \{ \text{calculus} \} & \\ \exists j \geq 0. R^j(s) \models \phi. & \end{aligned}$$

$$\begin{aligned} s \models G\phi & \\ \Leftrightarrow \{ \text{definition of } G \} & \\ s \models \neg F \neg \phi & \\ \Leftrightarrow \{ \text{using result of previous derivation} \} & \\ \neg[\exists j \geq 0. R^j(s) \models \neg \phi] & \\ \Leftrightarrow \{ \text{semantics of } \neg \phi \} & \\ \neg[\exists j \geq 0. \neg(R^j(s) \models \phi)] & \\ \Leftrightarrow \{ \text{predicate calculus} \} & \\ \forall j \geq 0. R^j(s) \models \phi. & \end{aligned}$$



1. $\phi \Rightarrow F\psi$: if initially (that is, in s) ϕ , then eventually ψ .
2. $G[\phi \Rightarrow F\psi]$: if ϕ then eventually ψ (for all successor states of s and the initial state s).
3. $GF\phi$: ϕ holds infinitely often.
4. $FG\phi$: eventually permanently ϕ .
5. $G[\phi \Rightarrow X\phi]$: if ϕ is valid in some state, then it is also valid in its successor state.
6. $G[\phi \Rightarrow G\phi]$: once ϕ , always ϕ .

$$\begin{aligned}
\text{Duality rules:} \quad & \neg G \phi \equiv F \neg \phi \\
& \neg F \phi \equiv G \neg \phi \\
& \neg X \phi \equiv X \neg \phi
\end{aligned}$$

$$\begin{aligned}
\text{Idempotency rules:} \quad & G G \phi \equiv G \phi \\
& F F \phi \equiv F \phi \\
& \phi U (\phi U \psi) \equiv \phi U \psi \\
& (\phi U \psi) U \psi \equiv \phi U \psi
\end{aligned}$$

$$\begin{aligned}
\text{Absorption rules:} \quad & F G F \phi \equiv G F \phi \\
& G F G \phi \equiv F G \phi
\end{aligned}$$

$$\text{Commutation rule:} \quad X(\phi U \psi) \equiv (X\phi) U (X\psi)$$

$$\begin{aligned}
\text{Expansion rules:} \quad & \phi U \psi \equiv \psi \vee [\phi \wedge X(\phi U \psi)] \\
& F \phi \equiv \phi \vee X F \phi \\
& G \phi \equiv \phi \wedge X G \phi
\end{aligned}$$

Definition 18. (Syntax of computation tree logic)

For $p \in AP$ the set of CTL-formulas is defined by:

$$\boxed{\phi ::= p \mid \neg \phi \mid \phi \vee \phi \mid EX \phi \mid E[\phi U \phi] \mid A[\phi U \phi].}$$

Four temporal operators are used:

- EX (pronounced “for some path next”)
- E (pronounced “for some path”)
- A (pronounced “for all paths”) and
- U (pronounced “until”).

$$\begin{aligned}
EG \phi &\equiv \neg AF \neg \phi \\
AG \phi &\equiv \neg EF \neg \phi \\
AX \phi &\equiv \neg EX \neg \phi.
\end{aligned}$$

For instance, we have

$$\begin{aligned}
&\neg A(F \neg \phi) \\
&\Leftrightarrow \{ A\psi \equiv \neg E \neg \psi \} \\
&\quad \neg \neg E \neg (F \neg \phi) \\
&\Leftrightarrow \{ G\psi \equiv \neg F \neg \psi; \text{calculus} \} \\
&EG \phi.
\end{aligned}$$

$EG \phi$ is pronounced “potentially always ϕ ”, $AG \phi$ is pronounced “invariantly ϕ ” and $AX \phi$ is pronounced “for all paths next ϕ ”. The operators E and A bind equally strongly and have the highest precedence among the unary operators. The binding power of the other operators is identical to that of linear temporal logic PLTL. Thus, for example, $(AG p) \Rightarrow (EG q)$ is simply denoted by $AG p \Rightarrow EG q$, and should not be confused with $AG(p \Rightarrow EG q)$.

Definition 19. (CTL-Model)

A CTL-model is a triple $\mathcal{M} = (S, R, Label)$ where

- S is a non-empty set of states,
- $R \subseteq S \times S$ is a total relation on S , which relates to $s \in S$ its possible successor states,
- $Label : S \longrightarrow 2^{AP}$, assigns to each state $s \in S$ the atomic propositions $Label(s)$ that are valid in s .

$\mathcal{M}$ is also known as a *Kripke structure*³, since Kripke used similar structures to provide a semantics for modal logic, a kind of logic that is closely related to temporal logic (Kripke, 1963).

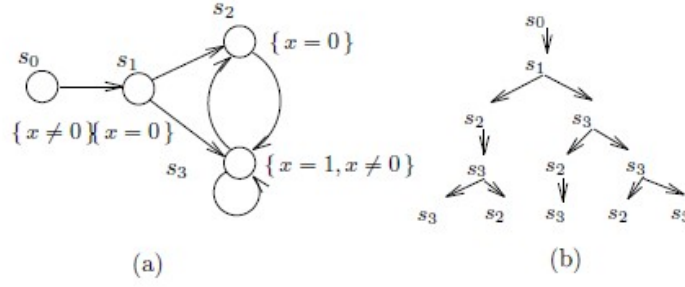


Figure 3.1: An example of a CTL-model and a (prefix of) one of its infinite computation trees

Definition 20. (Path)

A *path* is an infinite sequence of states $s_0 s_1 s_2 \dots$ such that $(s_i, s_{i+1}) \in R$ for all $i \geq 0$.

Let $\sigma \in S^\omega$ denote a path (of states). For $i \geq 0$ we use $\sigma[i]$ to denote the $(i+1)$ -th element of σ , i.e. if $\sigma = t_0 t_1 t_2 \dots$ then $\sigma[i] = t_i$, where t_i is a state.

Definition 21. (Set of paths starting in a state)

The set of paths starting in state s of the model $\mathcal{M}$ is defined by

$$P_{\mathcal{M}}(s) = \{ \sigma \in S^\omega \mid \sigma[0] = s \}.$$

Definition 22. (Semantics of CTL)

Let $p \in AP$ be an atomic proposition, $\mathcal{M} = (S, R, Label)$ be a CTL-model, $s \in S$, and ϕ, ψ be CTL-formulas. The satisfaction relation $\models$ is defined by:

$$\begin{aligned} s \models p & \quad \text{iff } p \in Label(s) \\ s \models \neg \phi & \quad \text{iff } \neg (s \models \phi) \\ s \models \phi \vee \psi & \quad \text{iff } (s \models \phi) \vee (s \models \psi) \\ s \models EX \phi & \quad \text{iff } \exists \sigma \in P_{\mathcal{M}}(s). \sigma[1] \models \phi \\ s \models E[\phi U \psi] & \quad \text{iff } \exists \sigma \in P_{\mathcal{M}}(s). (\exists j \geq 0. \sigma[j] \models \psi \wedge (\forall 0 \leq k < j. \sigma[k] \models \phi)) \\ s \models A[\phi U \psi] & \quad \text{iff } \forall \sigma \in P_{\mathcal{M}}(s). (\exists j \geq 0. \sigma[j] \models \psi \wedge (\forall 0 \leq k < j. \sigma[k] \models \phi)). \end{aligned}$$

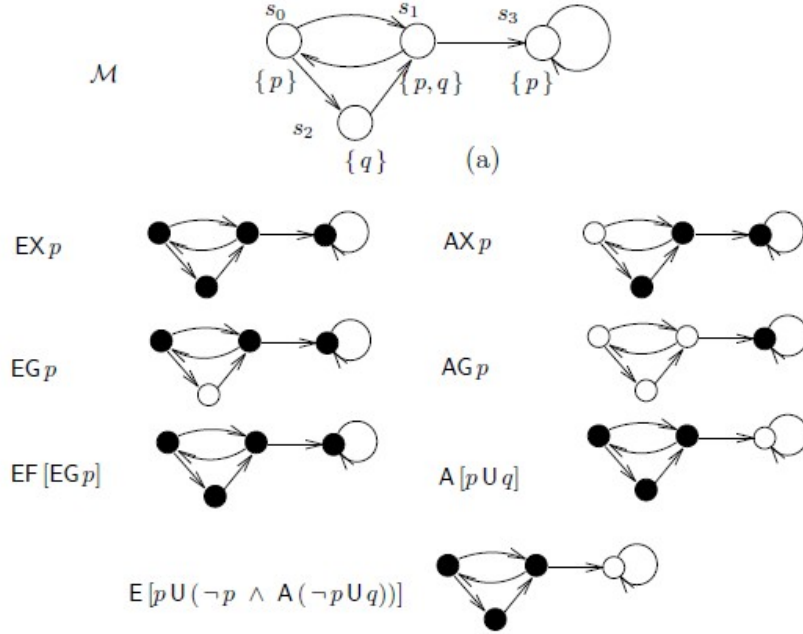


Figure 3.2: Interpretation of several CTL-formulas in an example of a model

- $AG\ p$ is only valid for s_3 since its only path, s_3^ω , always visits a state in which p holds. For all other states it is possible to have a path which contains s_2 , which does not satisfy p . So, for these states $AG\ p$ is not valid.
- $EF\ [EG\ p]$ is valid for all states, since from each state a state (either s_0 , s_1 or s_3) can be reached eventually from which some computation starts along which p is globally valid.
- $A\ [p\ U\ q]$ is not valid in s_3 since its only computation (s_3^ω) never reaches a state for which q holds. For all other states this is, however true and in addition, the proposition p is valid before q is valid.
- Finally, $E\ [p\ U\ (\neg p \wedge A(\neg p\ U\ q))]$ is not valid in s_3 , since from s_3 a q -state can never be reached. For the states s_0 and s_1 the property is valid, since state s_2 can be reached from these states via a p -path, $\neg p$ is valid in s_2 , and from s_2 all possible paths satisfy $\neg p\ U\ q$, since s_2 is a q -state. For instance, for state s_0 the path $(s_0\ s_2\ s_1)^\omega$ satisfies $p\ U\ (\neg p \wedge A(\neg p\ U\ q))$ since $p \in \text{Label}(s_0)$, $p \notin \text{Label}(s_2)$ and $q \in \text{Label}(s_1)$. For state s_2 the property is valid since p is invalid in s_2 and for all paths starting at s_2

PLTL		$\phi ::= p \mid \neg \phi \mid \phi \vee \phi \mid X\phi \mid \phi U \phi$
CTL	state formulas	$\phi ::= p \mid \neg \phi \mid \phi \vee \phi \mid E\psi$
	path formulas	$\psi ::= \neg \psi \mid X\phi \mid \phi U \phi$
CTL*	state formulas	$\phi ::= p \mid \neg \phi \mid \phi \vee \phi \mid E\psi$
	path formulas	$\psi ::= \phi \mid \neg \psi \mid \psi \vee \psi \mid X\psi \mid \psi U \psi$

Table 3.1: Summary of syntax of PLTL, CTL and CTL*

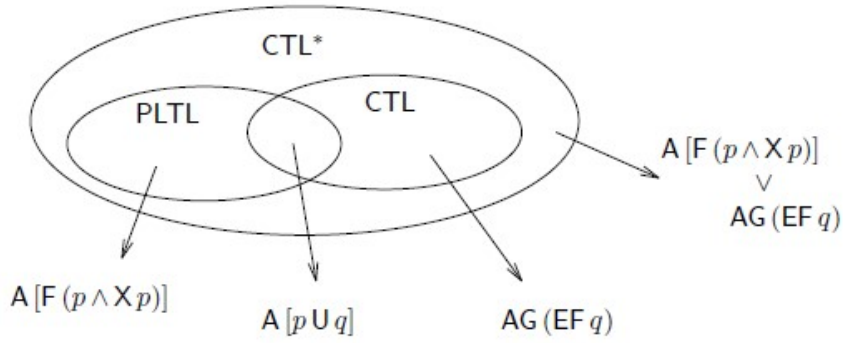


Figure 3.3: Relationship between PLTL, CTL and CTL*

The syntax of CTL requires that the linear temporal operators X , F , G , and U are immediately preceded by a path quantifier E or A . If this restriction is dropped, then the more expressive branching temporal logic CTL* (Clarke and Emerson, 1981) is obtained. The logic CTL* permits an arbitrary PLTL-formula to be preceded by either E or A . It contains, for instance, $E[p \wedge Xq]$ and $A[Fp \wedge Gq]$, formulas which are not syntactical terms of CTL. CTL* can therefore be considered as *the* branching counterpart of PLTL since each PLTL sub-formula can be used in a CTL*-formula. The precise relationship between PLTL, CTL and CTL* will be described in Section 3.3. We do not consider model checking CTL* in these lecture notes, since model checking of CTL* is of intractable complexity: the model checking problem for this logic is PSPACE-complete in the size of the system specification (Clarke, Emerson and Sistla, 1986). We therefore consider CTL for which more efficient model checking algorithms do exist. Although CTL does not possess the full expressive power of CTL*, various (industrial) case studies have shown that it is frequently sufficiently powerful to express most required properties.

Bibliografie:

Joost-Pieter Katoen, Concepts, Algorithms and Tools for Model Checking, 1999